

Cypher: AI Copilot for Integrated File Knowledge

Cypher is an Industrial AI copilot designed for integrated file knowledge. Industrial systems have a massive amount of files, including manuals, procedures, log files, etc. When someone needs to find specific information, they usually have to rely on keyword-based searching, which is highly time-consuming and often inaccurate.

Cypher solves this. It automatically ingests every document type(pdf,images,videos, audio ppt etc..), extracts its entities and relationships into a knowledge graph, and lets your team ask complex questions in plain English. In short, it intelligently connects the actual content hidden inside your files.

The architecture operates on a **layer-based workflow**:

1. **Ingestion Layer:** Extracts, chunks, and processes content from files, storing it structurally across multiple databases.
 2. **Retrieval & Generation Layer:** When a user asks a question, it fetches the most relevant context from those databases and feeds it to a large language model (LLM) to generate a precise, cited answer.
-

Both layers works with qdrant and then neo4j

1. Ingestion Layer (Processing the Files)

This process starts when a user selects a folder path to be tracked. The agent scans the folder and tracks new or changed files (tracked by hashing).

FOR QDRANT

- **Extraction (The Dispatcher):** First, the file is routed by its extension to a specific processor to extract its text, tables, and metadata:
 - **PDFs/Office docs:** IBM Docling
 - **Images/Scans:** Tesseract OCR (If the text is too sparse, it falls back to **Qwen-VL** to visually describe the image)
 - **Audio/Video:** OpenAI Whisper / ffmpeg
 - **Excel/CSV:** Pandas (creates statistical summaries)
- **Chunking:** The extracted text is then passed to a chunker. Because LLMs can't read entire books at once, the text is broken into "chunks" of ~1,000 characters. The chunker is sentence-aware and leaves a 150-character overlap between chunks so no context is lost.
- **Vectorization:** Each text chunk is passed to the **BAAI/bge-base-en-v1.5** embedding model. This model converts the text into mathematical vectors (embeddings), which are then stored in **Qdrant** (a vector database).

FOR NEO4J

- **Entity Extraction (Knowledge Graph):** While vectorization happens, a smaller LLM (**Qwen2.5-3B**) scans the document using 2000-character sliding windows. It is instructed to look for strict industrial concepts (Entities: EQUIPMENT, FAILURE, PROCEDURE / Relationships: PART_OF, HAS_FAILURE). It merges these concepts into standard IDs and stores them as a connected graph in **Neo4j**.

(Note: During all of this, the exact file path is saved alongside every chunk and graph node as its unique identity).

Type	Library / model	Fallback
PDF	IBM Docling (DocumentConverter)	Tesseract OCR for scanned/image PDFs
Image	Tesseract OCR first (cheap)	if text < 30 chars → Qwen-VL vision describes it
Office (.docx/.pptx/.odt)	Docling (lazy-loaded)	plain read for .txt/.html
Excel/CSV	pandas	sheets > 150 rows → statistical summary (schema + min/max/mean/std + head/tail) instead

Type	Library / model	Fallback
Audio	OpenAI Whisper (base)	of row-by-row —
Video	ffmpeg → Whisper (transcript) + Qwen-VL (keyframes)	—
Email	stdlib email (.eml)	extract-msg (.msg, optional)

Then the pipeline runs:

2. Retrieval Layer (Answering the User)

When a user asks a question in the UI, the system runs a multi-step retrieval pipeline to gather the right context before answering:

FOR QDRANT

- **Vector Retrieval (Qdrant):** The user's prompt is converted into an embedding using the same **BAAI** model. The system searches Qdrant and purposely "over-fetches" the top 15 most mathematically similar text chunks.
- **Reranking:** Those 15 chunks, along with the user's prompt, are given to **cross-encoder/ms-marco-MiniLM-L-6-v2**. This model acts as a highly accurate judge. It deeply evaluates the chunks against the prompt and trims the list down to the absolute best **Top 5 results**.

FOR NEO4J

- **Graph Retrieval (Neo4j):** Simultaneously, the system looks for known entity names in the user's prompt. It queries **Neo4j** and performs a subgraph traversal to pull related entities, relationships, and the source documents that mention them.
 - **Context Fusion:** The Top 5 Qdrant chunks and the Neo4j graph facts are merged into one large context block. Crucially, every single piece of information is labeled with its **exact source filename**.
 - **Generation & Verification:** This perfectly assembled context block is handed to the main LLM (**Qwen3VL-8B**). The system strictly prompts the model to answer the question *only* using the provided context and mandates that it cites the exact filename in `[square brackets]`. Finally, the backend verifies that the citations are real before streaming the answer to the UI.
-

AI Models Used

- **BAAI/bge-base-en-v1.5:** For creating embeddings (vectors) during ingestion and retrieval.
- **cross-encoder/ms-marco-MiniLM-L-6-v2:** For scoring/reranking the over-fetched Qdrant results against the user's prompt.
- **Qwen3VL-8B-Instruct-Q4_K_M.gguf:** The primary model. It is responsible for the final generation of answers and for visually describing complex images during ingestion or chat.
- **Qwen2.5-3B-Instruct-Q4_K_M.gguf:** A lightweight model used strictly in the background during file ingestion to extract entities and relationships for Neo4j.

Databases Used

- **Postgres (or SQLite):** The relational system of record. It stores file hashes, folders, chat sessions, and history.
- **Qdrant:** The vector database for storing and querying text chunk embeddings.
- **Neo4j:** The graph database for storing typed entities and their relationships.

DATABASES structure

Qdrant (vector store) — collection industrial_knowledge, 768-dim, cosine. Each point:

```
id: uuid
vector: [768 floats]
(normalized BGE embedding of the chunk)
payload: { text, user_id, metadata: { source: <file_path>, file_type, chunk_index,
... } }
```

Neo4j (knowledge graph) — nodes and typed edges, scoped per user:

(:Document {path, type, user_id})

(:Entity

{id, name, type, description, user_id})

-- MERGED per (id,user_id)

(:Document)-[:MENTIONS]->(:Entity)

(:Entity)-[:HAS_FAILURE|PART_OF|GOVERNED_BY|MEASURES|...]->(:Entity)

-- native

typed edges